Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Hashing and Encryption

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Hash Function Cryptography
- Symmetric Cryptography
- Asymmetric Cryptography
- Advantages, disadvantages and uses of the above techniques
- How all three are used in SSL/TLS

UNIVERSITY of HULL

Cryptographic Principles

We've looked at identifying threats and authenticating and authorising users

- In order to do this safely, and in order to continue to be safe once we're sure who we're connected to, we will need to accomplish two things:

  1. A way to keep data secure between the two endpoints so that only the client and server are able to access the data.

  2. A way to be certain that what has been sent to you has not been tampered with along its journey.

UNIVERSITY of HULL

Cryptographic Principles

Remember that the goal is to secure data from the moment that it leaves the server application to the moment that it is received by the client application.

For the most part we treat anything in between these two endpoints as untrusted.

It's much harder to secure data that exists in running code on a machine…

- Data that is in memory is easily accessible
- Hardware typically ***has*** to have access to unencrypted data
- Viruses and other malicious code could, possibly, cause data exposure

- With good antivirus software and physical security, we can usually prevent access.
- However, we'll discuss 'secure' key storage later.

UNIVERSITY of HULL

The Three Main Algorithm Types

There are 3 main types of cryptographic algorithm:

1. Cryptographic Hash Functions
   - One-way function, designed to turn data into a unique and unidentifiable string – called a hash.
   - The same data will always turn into the same hash
   - Used as a 'checksum'

2. Asymmetric Encryption
   - One-way encryption between two endpoints
   - Managed by Public/Private key pairs
   - Public key used for encryption, private key used for decryption
   - Therefore anyone can have the public key, only one party can have the private key

3. Symmetric Encryption
   - Two-way encryption between two endpoints
   - Both parties use the same key for both encryption and decryption

UNIVERSITY of HULL

Hash Function Properties

One-way

- It must be infeasible to backtrack in order to identify the plaintext (this is called preimage resistance)

Deterministic

- The hash of a plaintext should always be the same, once it has passed through the function

Quick

- The hash function should be fast to process

Avalanche Effect

- A small change to the plaintext should have a large change to the hash, such that it is impossible to tell that the plaintexts were similar

Collision Resistant

- It should be infeasible to find two different plaintexts that create the same hash

(this is called second preimage resistance)

Avalanche

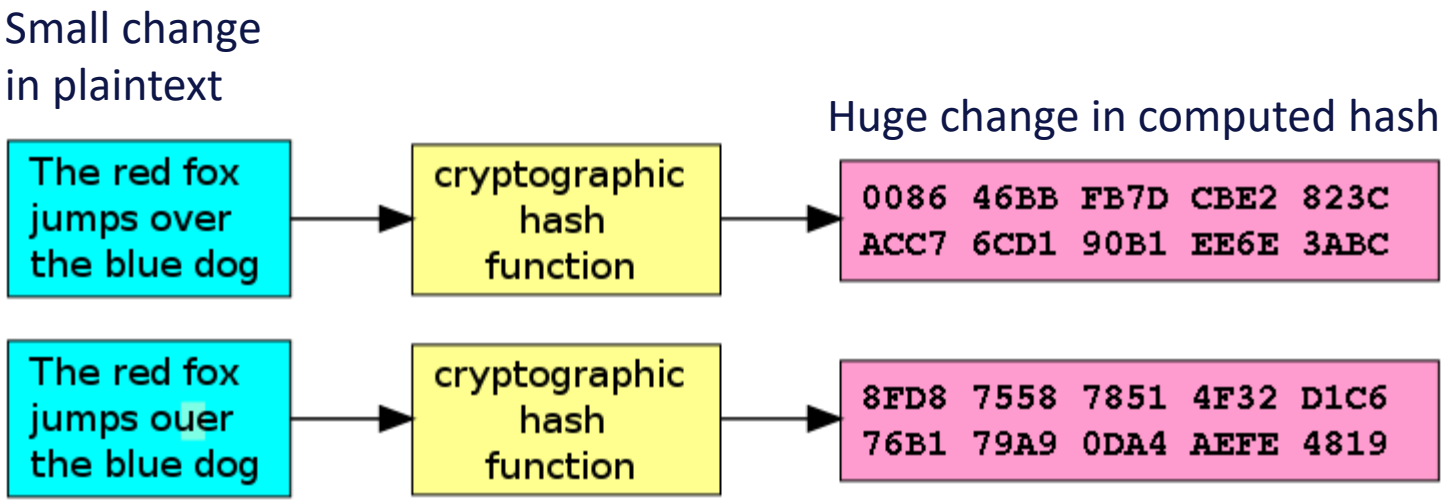


Image source: Wikipedia

Compression

A hash function will always have the same number of output bits, no matter how many input bits

That means that if you hash any message (with any number of bits) using a 128bit hash function you will get a 128bit hash

This makes storing hashes very easy as we know exactly how much space each will take up

Compression is done with fixed block sizes. The Merkle–Damgård construction (for example):

- Breaks down the plaintext into smaller blocks
- Operates a compression function on these blocks sequentially, combining the output of the previous function with the next block
- Pads the last block with information about how long the original plaintext was and some zeroes
- Produces a hash of the block size (or smaller through extra compression)

Compression actually helps with the avalanche effect

Using Hashes

Hashes are used for passwords and data checksums

When saving a password, the server doesn't want to store your password in plaintext because if it does and then the server is compromised, your password is compromised.

Instead…

- the server stores the hash and
- computes the hash once you have sent your password, then
- compares the hashes to determine if your password is correct

Two things to note:

- This process is useless to store any data that the server need to actually use/display (e.g. addresses). It's only a comparison technique!
- The password is often sent from the client un-hashed so must be encrypted
- Why is the password often sent un-hashed?

UNIVERSITY of HULL

Using Hashes

Why is the password often sent un-hashed?

So the client isn't made (trusted) to do the cryptographic work correctly

- Server's choice of hash function and server knows hash has been generated properly to avoid weaknesses

So security can be improved easily later by just updating the server

- Start using a new, more secure hash function for new accounts
- Update old accounts with new hash function when user logs in next time (and sends plaintext)

Because there is no benefit in making the client do the work…

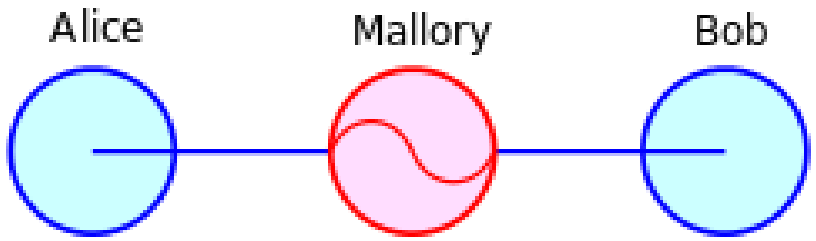
- If you send the password and an attacker gets hold of your plaintext password
  - Attacker can send your password and log in as you – you are compromised
- If you send the hash and an attacker gets hold of the hash – you are compromised
  - Attacker can send your hash and log in as you – you are compromised

UNIVERSITY of HULL

Using Hashes - Checksums

Bob sends Alice some data

Alice receives the data but doesn't know if what she has received is the same data Bob sent because someone who is sitting between Bob and Alice (Mallory) could have swapped out the data from what was originally sent to something malicious.



So instead, Bob sends Alice both the data and its computed hash. When the data arrives, Alice hashes it to check if the computed hash matches the hash that Bob sent. If Mallory changes the data, the hashes won't match and Alice knows it is unsafe.

Of course, if Mallory changes the data and the hash this process is useless so:

- Either the hash has to be sent using different means
- Or the data and/or the hash must be encrypted

UNIVERSITY of HULL

Image source: Wikipedia

Encryption

Unlike hashes, encryption is reversible

This is most useful when you want to send data to another party or if you want to store data and don't want it to be readable if it is stolen.

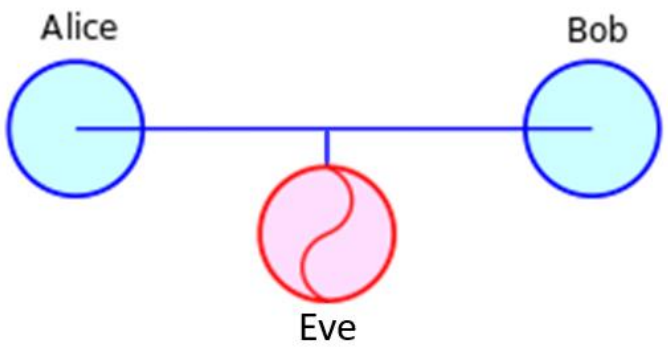
As we've seen, standard encryption comes in two normal forms:

1. Symmetric encryption
2. Asymmetric encryption

Symmetric Encryption

Bob is sending some private data to Alice.

Now, Eve is silently listening to the data. Bob and Alice cannot identify that Eve is intercepting and stealing the data.



Thankfully Bob and Alice have already agreed on a symmetric key and cryptographic function to use.

They both have the same key and can encrypt information before it leaves their application and as decrypt it as it comes in.

Eve can still intercept the data but, without the key, the data is unreadable nonsense

UNIVERSITY of HULL

Symmetric Encryption Principles

The main goals for a symmetric key encryption function:

Quick

- Should be fast. *Normally it's a balance between speed and security though*

Reversible

- The algorithm should be able to decrypt as well as encrypt, essentially by reversing the process

Key-Based Determinism

- Given the same key, the same plaintext should always produce the same encrypted message and vice versa

Inverse Resistance

- Given an encrypted message, or a set of encrypted messages, it should be infeasible to be able to identify the key used

Symmetric Cryptography Types

Block ciphers

– Encrypt plain text blocks

– e.g. DES, TripleDES, AES

Stream ciphers

– Encrypt plain text one bit / byte at a time

– e.g. RC4

We will look at the steps for AES, but we won't go into the maths in detail

- There are lots of online resources for this if you are interested

UNIVERSITY of HULL

Symmetric Cryptography

- *Advanced Encryption Standard (AES)*
  - Also known as *Rijndael Cipher*
- Block cipher
  - *128*, *192* or *256* bit keys and 128bit (16 byte) block sizes
  - *10* to *14* rounds of encryption
- Principles
  - **One round consists of four steps: SubBytes, ShiftRows, MixColumns, AddRoundKey**
  - **Uses:** Matrices*, byte substitution, Vector-matrix multiplication, XOR, circular left shifts*
  - Inverse is applied to reverse encryption, but can only be done if key is known.
- Properties
  - Key is *shared* between *A* and *B* - (K_{AB})
  - Encryption $\{M\}_{K_{AB}}$ - (Message protected by a shared, single key, known by A and B)
  - Only A and B can read the message
  - *B presumes* that the message is from *A* (since nobody else is supposed to know K_{AB})

UNIVERSITY of HULL

Image sources: Wikipedia

Block size

Our plaintext is split into blocks of bytes. For AES this is 16 bytes per block:

Block 1:

byte1	**byte5**	**byte9**	**byte13**
byte2	**byte6**	**byte10**	**byte14**
byte3	**byte7**	**byte11**	**byte15**
byte4	**byte8**	**byte12**	**byte16**

Block 2:

byte17	**byte21**	**byte25**	**byte29**
byte18	**byte22**	**byte26**	**byte30**
byte19	**byte23**	**byte27**	**byte31**
byte20	**Byte24**	**byte28**	**Byte32**

Etc…

The last block may need padding to make up the final 16 bytes.

Image sources: Wikipedia

Symmetric Cryptography

- ***SubBytes*** *(substitution matrix)*
- *A mathematically calculated substitution table and inverse substitution applied on a per-bit basis*

FROM:

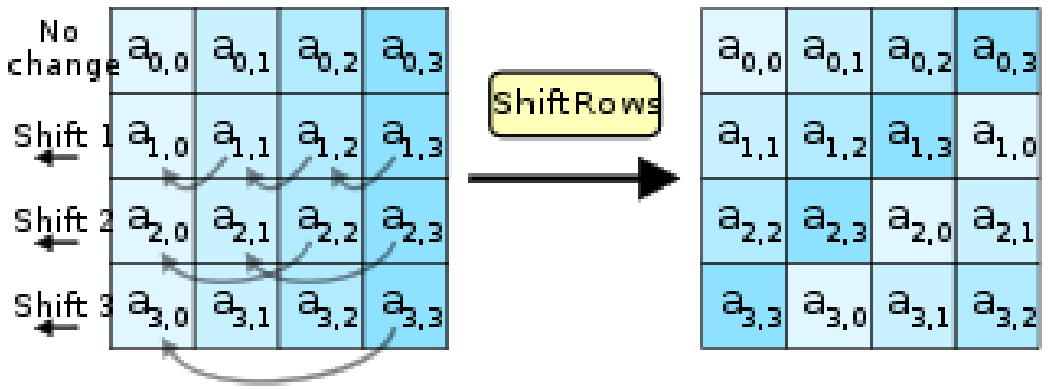
	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

TO:

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
10	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
20	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
30	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
40	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
50	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
60	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
70	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
80	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
90	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a0	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b0	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c0	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d0	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e0	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f0	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

Symmetric Cryptography

- ***ShiftRows*** *(prevents columns existing as smaller blocks which can be independently reversed)*



byte1	**byte5**	**byte9**	**byte13**
byte2	**byte6**	**byte10**	**byte14**
byte3	**byte7**	**byte11**	**byte15**
byte4	**byte8**	**byte12**	**byte16**

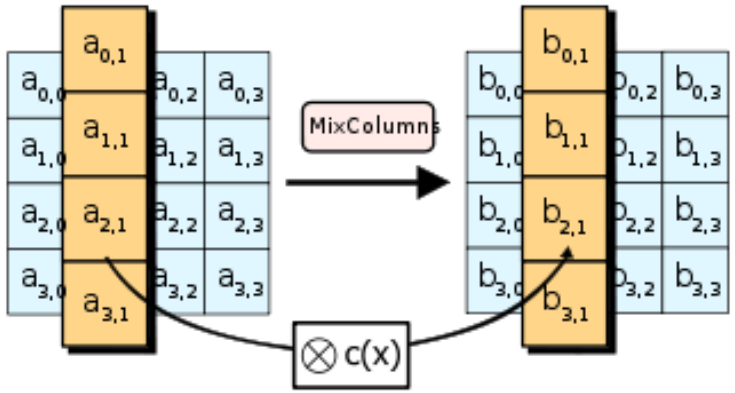
byte1	**byte5**	**byte9**	**byte13**
byte6	**byte10**	**byte14**	**byte2**
byte11	**byte15**	**byte3**	**byte7**
byte16	**byte4**	**byte8**	**byte12**

UNIVERSITY of HULL

Image sources: Wikipedia

Symmetric Cryptography

- ***MixColumns*** *(uses all input column bytes to create different output column bytes using XOR)*
- *Mathematically complex calculations that mix together the bytes in each column of the input*
- *Adds diffusion – if any bit in the plaintext block changes then (on average) half of the computed ciphertext should change (for this round)*



MixColumns Matrix Operation

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \bullet \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \rightarrow$$

$$\begin{aligned} d_0 &= 2 \bullet b_0 \oplus 3 \bullet b_1 \oplus 1 \bullet b_2 \oplus 1 \bullet b_3 \\ d_1 &= 1 \bullet b_0 \oplus 2 \bullet b_1 \oplus 3 \bullet b_2 \oplus 1 \bullet b_3 \\ d_2 &= 1 \bullet b_0 \oplus 1 \bullet b_1 \oplus 2 \bullet b_2 \oplus 3 \bullet b_3 \\ d_3 &= 3 \bullet b_0 \oplus 1 \bullet b_1 \oplus 1 \bullet b_2 \oplus 2 \bullet b_3 \end{aligned}$$

Where d_0- d_3 are output bytes
and b_0-b_0 are input bytes
Numbers are a constant

- is polynomial multiplication using Galois field (see: finite field arithmetic)

UNIVERSITY of HULL

Image sources: Wikipedia

Symmetric Cryptography

- ***AddRoundKey*** *(uses a round key based on the symmetric key, XOR to produce output)*

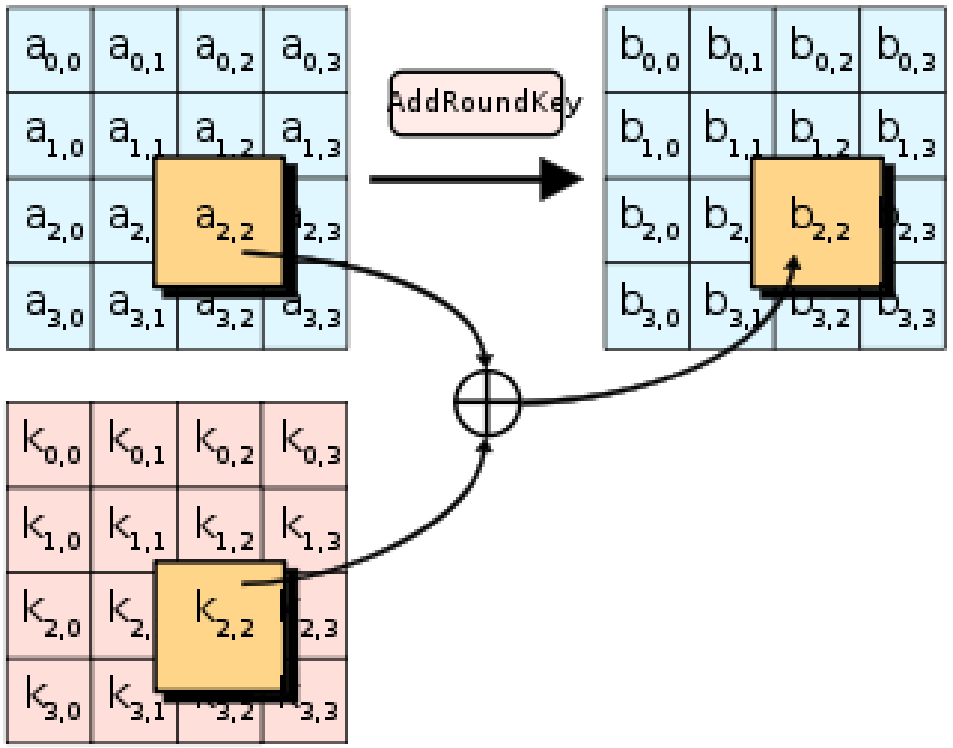


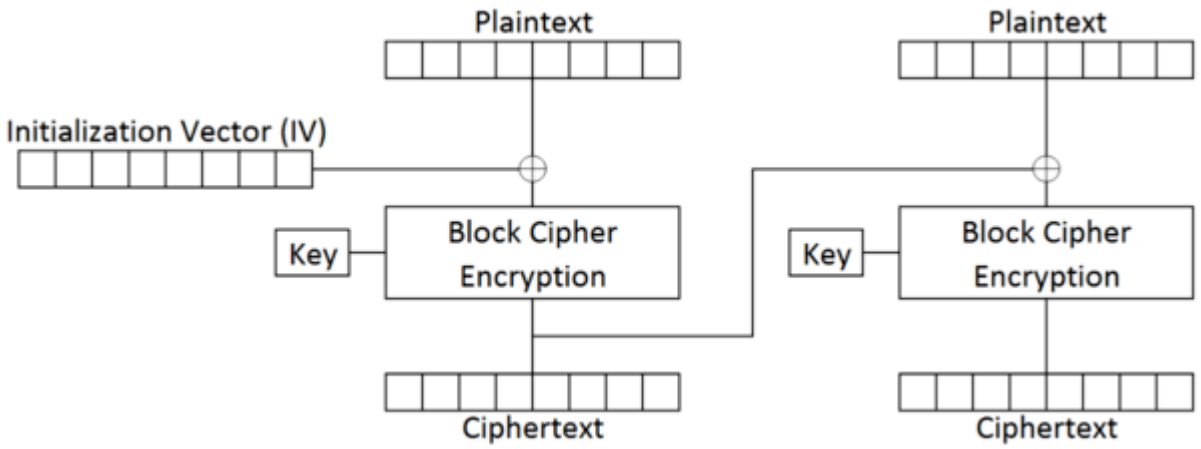
Image source: Wikipedia

Symmetric Encryption

Block ciphers are secure only for encryption of a single block

Cipher Block Chaining is needed: encryption/decryption of subsequent blocks is dependent on the outcome of encrypting/decrypting preceding blocks. This creates a cascade effect, where a small change in the plaintext causes a dramatic change in the ciphertext (even with the same key).

Ciphertext block is XORed with next block of plaintext. First block must be XORed with a known initialization vector (IV). This IV can be changed to produce a different output for the same plaintext:



UNIVERSITY of HULL

Symmetric Encryption Advantages/Disadvantages

Advantages

- Symmetric ciphers tend to be *fast*

Disadvantages

- Keys must be *pre-shared*
  - Symmetric cryptography does not support key exchange
- Every pair of participants have their *own pair of secret keys*
  - Potentially lots of keys to store if you have lots of connections
- Symmetric ciphers tend to be *less secure* than asymmetric alternatives

UNIVERSITY of HULL

Asymmetric Encryption

We just noted that keys must be pre-shared for symmetric cryptography to work.

This is fine if the applications Bob and Alice are running were developed by the same developer and the key can be preset during development - but this is rarely the case. Even in that situation, the longer Bob and Alice use the same key the higher the chance of Eve cracking the key.

Consider the situation we saw last session:

- Bob and Alice don't already know each other.
- Alice connects to Bob.
- Alice has private data to send to Bob (e.g. a password) but is wary of Eve and Mallory.
- How can Alice send encrypted data to Bob if they have no preconfigured keys?

The answer is asymmetric cryptography.

Asymmetric Encryption Principles

Public/Private Key Pair

- A public key for encryption, a private key for decryption

Speed

- Asymmetric cryptography is usually a lot slower than symmetric cryptography. Again, the balance is speed vs. security

Deterministic

- Given a public key, the matching private key should always reproduce the original plaintext

One-Way

- The public key cannot be used for decryption, the private key cannot be used for encryption

Key Disparity

- It should be infeasible to identify the private key from the public key and/or an encrypted message

Asymmetric Cryptography Example

1. Bob and Alice don’t already know each other.
2. But Bob already has created a public/private key pair.
3. Alice connects to Bob.
4. Alice has private data to send to Bob (e.g. a password) but is wary of Eve and Mallory.
5. Bob sends Alice his public key - unencrypted
6. Alice uses the key to encrypt her data, then sends the encrypted data to Bob
7. Bob now uses his private key to unencrypt the data and access the plaintext
8. Eve knows the public key and the encrypted message but, without the private key, cannot decrypt the data

UNIVERSITY of HULL

Asymmetric Cryptography

- **Encryption and decryption using different keys**

  Public key: can be known to everybody

  Private key: kept secret

- **Based on number-theoretical concepts**

  Factoring and modulus

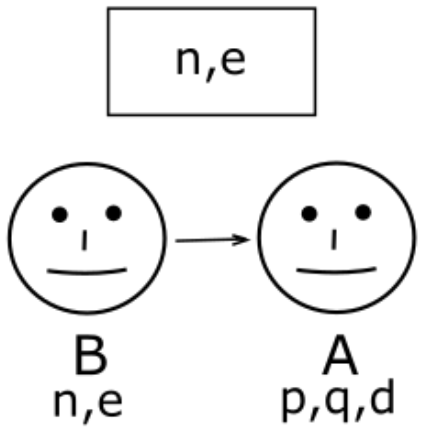
  Inverse of modular exponentiation (discrete logarithm)

- **Examples**

  RSA cryptosystem

  Diffie-Hellman Key Exchange

Asymmetric Cryptography



- RSA cryptosystem

  Ron **R**ivest, Adi **S**hamir, Len **A**dleman
- Parameters

  Public key: ***n, e***

  Private key: ***p, q, d***

  p, q are *large primes* such that $n = pq$

  φ(n): Euler Totient Function (number of ints <= n which are a co-prime with n)

  e: public encryption exponent: find co-prime of φ(n) (a co-prime shares no common factors except 1)

  d: private decryption exponent = e^{-1} mod φ(n)
- Encryption and decryption (M:Message, C:Cipher)

  Encryption (M->C) : $C = \{M\}K_{A,pub} = M^e \bmod n$

  Decryption (C->M) : $M = \{C\}K_{A,priv} = C^d \bmod n$
- Security

  1024 bits are not enough - 2048-bit keys are considered sufficient until 2030

See: https://simple.wikipedia.org/wiki/RSA_(algorithm)#A_working_example

Asymmetric Cryptography Advantages/Disadvantages

Advantages

- Asymmetric cryptography supports key exchange
- Public key can be disclosed to anybody
- Used in security certificates (SSL, TLS)
- Used in digital signature checking

Disadvantages

- One-way. Bob can't send Alice encrypted information without another key exchange.

- Asymmetric ciphers tend be much slower than symmetric

  Because of that, TLS/SSL allows Alice to create a **symmetric key** and share it with Bob using Bob's **asymmetric public key**. Then Bob and Alice use the faster symmetric cryptography to continue their conversation.

Asymmetric Cryptography Problems

Alone, this isn't enough. Remember Mallory?

Mallory wants to attack the connection, possibly by changing the public key.

Remember in the last lecture we saw this and talked about using certificates and trusted parties?

Certificate signing usually uses hashes so rather than signing the full data (as we saw last session) we usually sign a hash of the data.

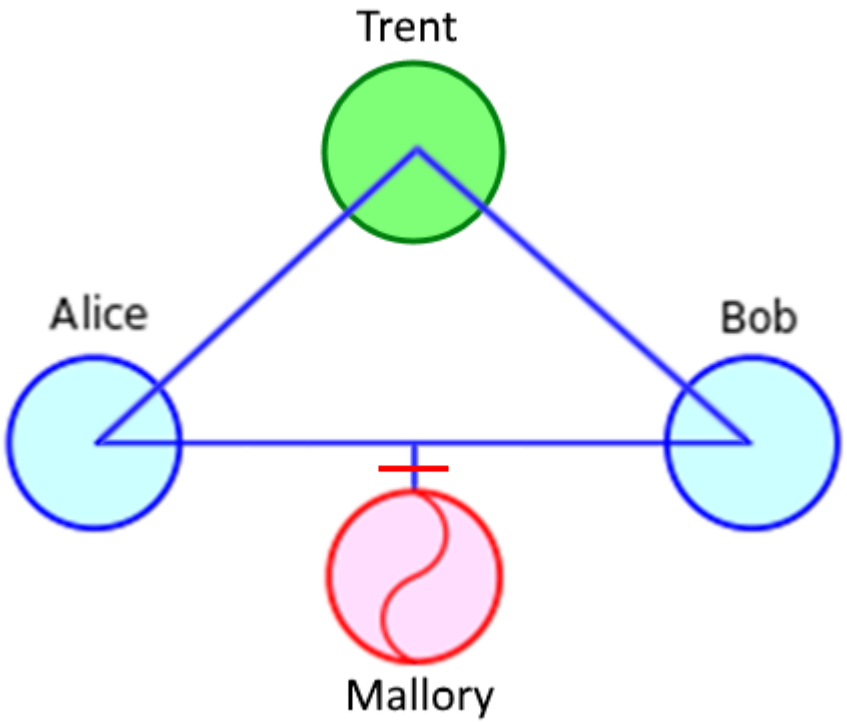
So SSL/TSL actually uses Hashes, Asymmetric and Symmetric Cryptography to work!

UNIVERSITY of HULL

Trusting Trent

- This is managed by one of a number of certificate authorities (Trent)
- These authorities are known to and trusted by applications(e.g. browsers: Firefox, Chrome, Edge, etc.)

- Alice trusts Trent and already has Trent's **public key**
- Bob has a certificate from Trent. Trent created it by signing a hash of Bob's **public key** using Trent's **private key**
- Bob sends his **public key** and the certificate to Alice.
- Alice can verify this certificate with Trent's **public key** (which she already knows)
- Alice now knows she definitely has Bob's correct **public key** and can send Bob a *symmetric key*, encrypted with Bob's **public key**.
- Mallory can not pretend to be Bob because Mallory doesn't know Trent's **private key** so can't create a valid certificate
- Mallory can not understand the data or change it without raising suspici



This all relies on Trent being pre-trusted

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Hash Function Cryptography
- Symmetric Cryptography
- Asymmetric Cryptography
- Advantages, disadvantages and uses of the above techniques
- How all three are used in SSL/TLS

In the next session we will be looking at performing Hashing and Encryption in C#

UNIVERSITY of HULL